The Data Refinery — Day 3/21 Learning Sprint

Day 3: SQL Core Essentials, Concepts & Solution Breakdown

Author: Prasad S Hiremath

Date: October 11, 2026

🧠 Concept 1: SQL Logical Execution Order (Engine Sequence)

While SQL queries are written starting with `SELECT`, database engines process clauses in a strict logical sequence. Mastering this prevents common column alias and filtering mistakes.

1. FROM / JOIN ➔ **2. WHERE** ➔ **3. GROUP BY** ➔ **4. HAVING** ➔ **5. SELECT** ➔ **6. ORDER BY**

⚖️ Concept 2: WHERE vs. HAVING Filtering

- **WHERE:** Filters raw row records *before* grouping occurs. Cannot contain aggregate functions like `SUM()` or `COUNT()`.
- **HAVING:** Filters aggregated group metrics *after* a `GROUP BY` clause calculates summaries. Operates purely on grouped outputs.

🔗 Concept 3: Complete Guide to SQL Join Types

INNER JOIN
Returns only matching records present in *both* left & right tables.

LEFT JOIN
Returns *all* left table rows + matching right table rows (fills missing with `NULL`).

RIGHT JOIN
Returns *all* right table rows + matching left table rows (fills missing with `NULL`).

FULL OUTER JOIN
Preserves *all* rows from both tables, populating `NULL` where matches fail.

CROSS JOIN
Generates a Cartesian product (Pairs every left row with every right row; N × M).

SELF JOIN
Joins a table onto itself to model hierarchical or sequential relationships.

💻 Day 3 Coding Challenge: Department Salary Spend

PRACTICAL CHALLENGE

Given the `employees` table below, write a standard SQL query to calculate the total salary spent per department, returning only departments where total spend exceeds **$100,000**.

emp_id	emp_name	department	salary
1	Rahul	Data	60000
2	Priya	Data	55000
3	Amit	DevOps	45000
4	Sneha	DevOps	40000
5	Vikram	Data	70000

THE DATA REFINERY

🛠️ Coding Challenge Solution & Error Breakdown

CODE SOLUTION

```sql
-- Production ANSI SQL Standard Solution SELECT department, SUM(salary) AS Total_salary_per_department FROM employees GROUP BY department HAVING SUM(salary) > 100000;
```

🔍 **Technical Analysis of Common Submission Errors:**

1. **Currency Formatting Error (`$100000`):** SQL numerical values must strictly consist of raw digits. Symbols like `$` or commas cause syntax compilation errors.
2. **Alias Evaluation in `HAVING`:** In ANSI SQL standards, `HAVING` is evaluated *before* `SELECT`. Therefore, column aliases defined in `SELECT` (e.g., `Total_salary_per_department`) do not exist yet when `HAVING` executes.

• *ANSI Engine Behavior (PostgreSQL, Oracle, SQL Server):* Rejects aliases in `HAVING`; requires `HAVING SUM(salary) > 100000`.

• *Modern Cloud Warehouses (Snowflake, BigQuery, MySQL):* Allow aliases in `HAVING` via compiler extensions, but using standard aggregate syntax ensures universal database compatibility.

MCQ 1: Logical Execution Priority

MCQ QUIZ

Which clause is evaluated **FIRST** by a database query planner?

A) `SELECT`

B) `WHERE`

C) `FROM` [CORRECT]

D) `GROUP BY`

Reasoning: The database engine must locate, open, and load the source dataset in memory (via `FROM` / `JOIN`) before it can evaluate filtering predicates (`WHERE`) or project columns (`SELECT`).

MCQ 2: Aggregate Filtering Clause

MCQ QUIZ

Which clause must be used to filter departmental groups **after** aggregation has been computed?

A) `WHERE`

B) `HAVING` [CORRECT]

C) `FILTER BY`

D) `QUALIFY`

Reasoning: Aggregate functions like `SUM()` or `COUNT()` are evaluated during the `GROUP BY` phase. `HAVING` is designed specifically to filter these aggregate outputs. (Note: `QUALIFY` filters window functions).

MCQ 3: Dataset Retention & Join Behavior

MCQ QUIZ

You join `customers` (50 rows) with `orders` (20 rows). Which JOIN type guarantees that **all records from both tables are retained**, filling unlinked fields with `NULL`?

A) `INNER JOIN`

B) `LEFT JOIN`

C) `CROSS JOIN`

THE DATA REFINERY

⭐ Bonus MCQ 4: NULL Handling in Aggregates

BONUS QUESTION

A table `bonus_payouts` has 5 rows where column `amount` contains: `[100, 200, NULL, 300, NULL]`. What are the outputs of `COUNT(*)` and `COUNT(amount)`?

A) `COUNT(*) = 3, COUNT(amount) = 3`

B) `COUNT(*) = 5, COUNT(amount) = 3` [CORRECT]

C) `COUNT(*) = 5, COUNT(amount) = 5`

D) `COUNT(*) = 3, COUNT(amount) = 5`

Reasoning: `COUNT(*)` counts total physical records regardless of content (5 rows). In contrast, `COUNT(column_name)` ignores `NULL` entries and counts only non-null values (3 non-null amounts: 100, 200, 300).

⭐ Bonus MCQ 5: The "Accidental Inner Join" Pitfall

BONUS QUESTION

Consider the query: `SELECT * FROM customers c LEFT JOIN orders o ON c.id = o.cust_id WHERE o.status = 'COMPLETED'`. What happens to customers who have no orders?

A) They are retained with `o.status = NULL`

B) They are filtered out, turning the LEFT JOIN into an INNER JOIN [CORRECT]

C) The query raises a syntax error

D) They are retained with `o.status = 'COMPLETED'`

Reasoning: Unmatched customers produce `NULL` for all right-table columns. Because the `WHERE` clause evaluates *after* the join, checking `NULL = 'COMPLETED'` yields `UNKNOWN` (false), silently discarding those customer rows! To retain unmatched left rows, move the status predicate into the `ON` join clause.

📋 Master Answer Key & Core Topic Summary

Exercise	Core Concept	Correct Key / Production Standard
Coding Challenge	Group Aggregation & Dialect Rules	**HAVING SUM(salary) > 100000**
MCQ 1	Logical Query Execution Order	**C (FROM Clause)**
MCQ 2	Post-Grouping Filter Clause	**B (HAVING Clause)**
MCQ 3	Full Dataset Retention	**D (FULL OUTER JOIN)**
Bonus MCQ 4	NULL Handling in Aggregates	**B (COUNT(*) = 5, COUNT(col) = 3)**
Bonus MCQ 5	LEFT JOIN Predicate Placement	**B (WHERE predicate converts LEFT JOIN to INNER JOIN)**

THE DATA REFINERY